ТОО «Колледж Хекслет»

КУРСОВАЯ РАБОТА

на тему: « Разработка интерактивного веб-приложения для систематизации и учета цифрового контента "Мемология" »

по модулю: «Прикладное программирование»

Специальность: 06130100 Программное обеспечение (по видам)

Квалификация: 4S06130105 Техник информационных систем

Выполнил(-а): студент 2 курса

группы 11 ТИС/24

Кадралиев Тамирлан Сергеевич

Допущен(-а) к защите «___»_____ 20__г.

Преподаватель: Аманжанов Х.А.

(подпись)

Защитил(-а) с оценкой:

_____

(буква) (цифр.экв.) (балл)

Алматы 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ: РАЗРАБОТКА ИНТЕРАКТИВНОГО ВЕБ-ПРИЛОЖЕНИЯ «МЕМОЛОГИЯ»	4
1.1 Анализ предметной области веб-приложения «Мемология»	4
1.2 Проектирование интерфейса приложения «Мемология»	5
1.3 Выбор технологий для разработки «Мемология»	6
2 ПРАКТИЧЕСКАЯ ЧАСТЬ: СОЗДАНИЕ ВЕБ-ПРИЛОЖЕНИЯ «МЕМОЛОГИЯ»	7
2.1 Реализация HTML-разметки приложения	7
2.2 Реализация CSS-стилей приложения	8
2.3 Реализация JavaScript-функционала приложения	9
2.4 Тестирование приложения на соответствие требованиям	10
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12

ВВЕДЕНИЕ

В современном информационном обществе цифровой контент, и в частности интернет-мемы, стали неотъемлемой частью массовой культуры и повседневной коммуникации. Они эволюционировали от простых текстовых шуток до сложных многоуровневых визуальных форматов, отражающих дух различных эпох (от зарождения интернета в 90-х до эпохи постиронии 2020-х годов). Однако, несмотря на огромные объемы потребляемой пользователями визуальной информации, существует острая нехватка удобных инструментов для ее структурирования, архивации и систематизации.

Целью данной курсовой работы является разработка адаптивного интерактивного веб-приложения «Мемология» - персонального веб-каталога для учета, коллекционирования и классификации интернет-наследия с функциями локального сохранения данных и отслеживания прогресса изучения материалов.

Актуальность проекта обусловлена несколькими факторами. Во-первых, наблюдается растущая потребность пользователей в организации хаотичного потока цифровой информации и создании личных структурированных баз данных. Во-вторых, разработка веб-приложений без использования тяжеловесных фреймворков (на нативном JavaScript) позволяет создавать максимально легковесные, быстрые и независимые клиентские решения, не требующие серверной обработки для базового функционала. В-третьих, стилизация интерфейса под ретро-операционные системы (скевоморфизм, пиксельная графика) является актуальным трендом в современном веб-дизайне, что делает проект не только технически функциональным, но и эстетически привлекательным.

Практическая значимость проекта заключается в создании полностью готового к эксплуатации веб-приложения, которое может быть использовано не только в качестве учебного примера архитектуры клиентских приложений на чистом JavaScript, но и как реальный пользовательский инструмент для каталогизации любого визуального контента. Приложение демонстрирует эффективность использования нативных веб-технологий для решения современных задач пользовательского интерфейса.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ: РАЗРАБОТКА ИНТЕРАКТИВНОГО ВЕБ-ПРИЛОЖЕНИЯ «МЕМОЛОГИЯ»

1.1 Анализ предметной области веб-приложения «Мемология»

Анализ предметной области является фундаментальным этапом разработки программного обеспечения. Он позволяет точно определить проблему, потребности аудитории и функциональные рамки будущего продукта. В рамках данной курсовой работы предметной областью выступает процесс коллекционирования, учета и систематизации цифрового визуального контента на примере интернет-мемов.

Современная цифровая культура генерирует огромные объемы информации. Интернет-мемы давно перестали быть просто смешными картинками. Они превратились в единицы культурной информации, которые отражают исторические эпохи, социальные тренды и специфический юмор различных поколений. Пользователи ежедневно сохраняют десятки изображений в галереи своих смартфонов или в раздел избранных сообщений в мессенджерах. Такой подход приводит к хаотичному скоплению файлов, среди которых становится невозможно быстро найти нужный контент.

Целевая аудитория разрабатываемого проекта включает в себя молодых людей в возрасте от 15 до 35 лет, студентов, активных пользователей социальных сетей и специалистов в области цифровых коммуникаций. Для этой группы пользователей характерна потребность в структурировании потребляемой информации и интерес к геймификации рутинных процессов.

Обзор существующих аналогов показал, что в сети присутствуют крупные энциклопедические ресурсы (например сайты Know Your Meme или Memepedia). Однако их главный недостаток заключается в отсутствии персонализации. Это просто справочники, которые не позволяют пользователю создать свою личную базу данных, сортировать контент по собственному желанию или отмечать изученные материалы.

Разрабатываемое приложение «Мемология» призвано заполнить эту нишу. Оно объединяет в себе функции справочного каталога по историческим эпохам интернета и личного интерактивного трекера, где пользователь может вести учет любимого контента.

1.2 Проектирование интерфейса приложения «Мемология»

Проектирование пользовательского интерфейса (UI) и пользовательского опыта (UX) проводилось с учетом современных требований к удобству использования и тематики проекта. Главной визуальной концепцией был выбран ретро-стиль, отсылающий к операционным системам конца 90-х годов (в частности Windows 95 и 98) и эпохе раннего интернета (Web 1.0). Подобный дизайн вызывает чувство ностальгии и идеально перекликается с сутью приложения, направленного на сохранение цифрового наследия.

Архитектура веб-приложения включает в себя несколько ключевых разделов, доступ к которым осуществляется через единое навигационное меню. Это обеспечивает логическую связность и позволяет пользователю интуитивно понимать структуру сайта. Главная страница выполняет роль личного кабинета и базы данных, страница формы служит для ввода новой информации, а страницы эпох несут образовательный характер.

Цветовая схема базируется на классических высококонтрастных цветах старых операционных систем:

- Фоновый цвет страниц задан в классическом оттенке морской волны (#008080).
- Основные контейнеры и окна имеют светло-серую заливку (#c0c0c0) с характерными объемными рамками, имитирующими выдавленный интерфейс.
- Текст выводится строгим черным цветом, а акцентные элементы оформления (кнопки и счетчики) выделены белым, синим и неоново-зеленым цветами.

Особое внимание при проектировании было уделено карточкам контента. Каждая карточка в коллекции имеет четкую иерархию: сверху располагается изображение, под ним находится метка категории (эпохи), далее следует название. В самом низу карточки размещена панель действий с чекбоксом для смены статуса и кнопкой удаления. Такое расположение элементов минимизирует когнитивную нагрузку на пользователя и делает работу с интерфейсом максимально предсказуемой. А также очень правильной.

1.3 Обоснование выбора технологий (HTML5, CSS3, JavaScript)

Технологический стек для разработки веб-приложения формировался на основе технического задания. Ключевым требованием являлся отказ от использования сторонних JavaScript-фреймворков и библиотек. Приложение должно было быть реализовано исключительно на базовых нативных веб-технологиях.

Для создания структуры и разметки страниц использован стандарт HTML5. Применение семантических тегов (header, nav, main, section, footer, article) позволило сделать код более читаемым и логичным как для разработчика, так и для браузерных алгоритмов. Форма добавления данных построена с использованием стандартных элементов ввода, при этом атрибуты обязательного заполнения намеренно исключены из HTML-кода для переноса всей логики проверок на сторону скриптов.

За визуальное оформление и адаптивность отвечает язык таблиц стилей CSS3. Для построения сетки карточек на главной странице применен модуль CSS Grid Layout, который позволяет легко выстраивать элементы в ровные ряды с автоматическим переносом на новые строки. Для выравнивания элементов внутри навигации и внутри самих карточек используется технология Flexbox. Адаптивность интерфейса под мобильные устройства и планшеты реализована посредством медиазапросов (@media). Они позволяют динамически изменять размеры шрифтов, ширину блоков и направление выравнивания в зависимости от ширины экрана устройства пользователя (в диапазоне от 320px до 1920px).

Программная логика приложения написана на стандарте JavaScript ES6+. Язык отвечает за интерактивность интерфейса: обработку кликов по кнопкам, перехват отправки форм, валидацию введенных данных и динамическую генерацию HTML-кода карточек.

Для сохранения пользовательских данных выбрана технология LocalStorage. Это свойство глобального объекта window, которое позволяет веб-приложениям сохранять данные в формате ключ-значение прямо в браузере пользователя. Выбор LocalStorage обусловлен тем, что данные сохраняются локально, не требуют подключения к серверной базе данных.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ: СОЗДАНИЕ ВЕБ-ПРИЛОЖЕНИЯ «МЕМОЛОГИЯ»

2.1 Реализация HTML-разметки приложения

Реализация пользовательского интерфейса началась с создания базового HTML-каркаса. В отличие от концепции одностраничных приложений (SPA), проект «Мемология» построен на многостраничной архитектуре. Это позволило физически разделить логику работы базы данных и информационные блоки различных эпох. В состав проекта входят файлы главной страницы (index.html), страницы формы (form.html), информационной страницы (about.html) и страниц-категорий (2000-s.html, 2010-s.html, modern.html).

Для построения структуры документов использован стандарт семантической разметки HTML5. Основные зоны интерфейса выделены с помощью тегов header (шапка сайта), nav (навигационное меню), main (основной контент) и footer (подвал сайта). Такое разделение помогает браузерам и программам экранного доступа правильно интерпретировать структуру страницы.

На главной странице (index.html) организована зона динамического вывода контента. В HTML-коде прописан пустой контейнер (div с идентификатором memes-container), в который посредством JavaScript встраиваются карточки элементов. Также предусмотрен блок статистики, содержащий элемент span для вывода актуального количества сохраненных записей.

Страница добавления контента (form.html) содержит тег form с уникальным идентификатором. Внутри формы расположены поля ввода: текстовое поле (input type="text") для названия, выпадающий список (select) для выбора эпохи и текстовое поле для ссылки на изображение. Согласно требованиям задания, атрибут required не применялся ни к одному из полей, поскольку процесс проверки заполненности был полностью перенесен на уровень языка JavaScript.

Для всех форм в html без исключений он был перенесен для более удобной верстки

2.2 Разработка адаптивных CSS-стилей и UI-дизайна

Визуальное оформление проекта реализовано с помощью каскадных таблиц стилей (файл style.css). Для обеспечения уникального дизайна в стиле операционных систем 90-х годов активно применялись свойства рамок (border) с эффектами inset и outset, а также строгая палитра цветов на основе серого, черного и белого оттенков. Для имитации старых шрифтов подключены гарнитуры "Press Start 2P" и "Jersey 10" через сервис Google Fonts.

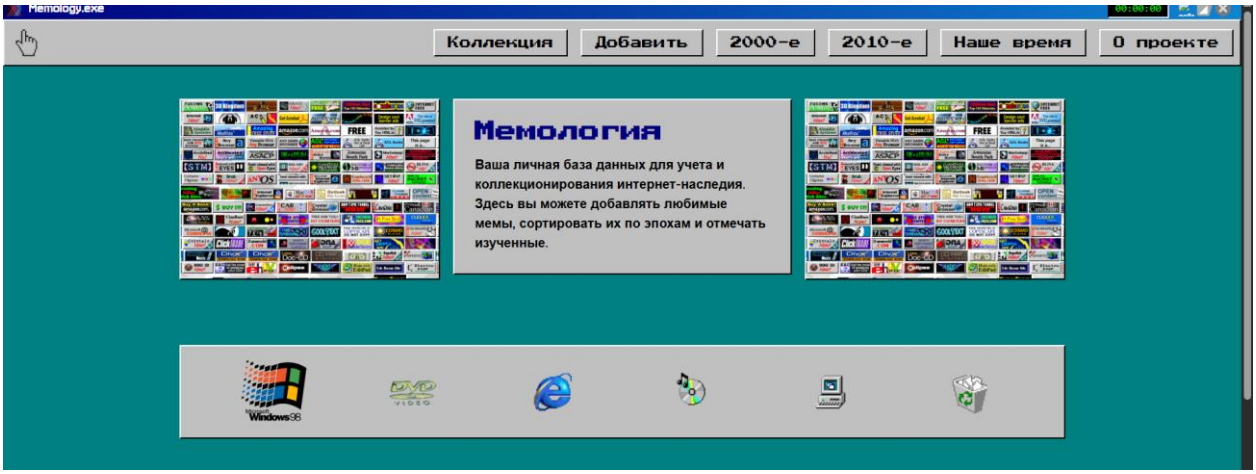

Рисунок 1 - Стиль старых сайтов

Организация элементов на странице выполнена с использованием двух современных спецификаций компоновки: Flexbox и CSS Grid. Технология Flexbox применена для выравнивания элементов навигации, шапки и подвала сайта. Она позволяет легко распределять свободное пространство между кнопками и выравнивать их по центру вертикальной оси. Технология CSS Grid использована для создания адаптивной сетки карточек на главной странице. Свойство grid-template-columns с параметрами repeat(auto-fill, minmax(300px, 1fr)) позволяет карточкам автоматически перестраиваться: если ширина экрана уменьшается, карточки выстраиваются в меньшее количество колонок, вплоть до одной колонки на мобильных устройствах.

Особое внимание в файле стилей уделено модификаторам состояния. Кнопки имеют псевдоклассы :hover (при наведении курсора) и :active (при нажатии), что создает эффект физического вдавливания кнопки. Для карточек реализован класс learned, который применяется при активации чекбокса "Изучено". Этот класс обесцвечивает изображение фильтром grayscale(100%) и зачеркивает текст заголовка.

Адаптивность веб-приложения (responsive web design) реализована через медиазапросы (@media). Контрольные точки (breakpoints) установлены на ширине 1024px, 768px и 480px. При уменьшении ширины экрана до 768px

(разрешение планшетов и смартфонов) навигационное меню перестраивается из горизонтальной строки в компактный блок, уменьшаются размеры заголовков, а многоколоночные текстовые блоки выстраиваются друг под другом. Это гарантирует корректное отображение контента на устройствах с шириной экрана от 320px.

2.3 Реализация JavaScript-логики и работы с LocalStorage

Программная логика приложения разделена на три независимых модуля, каждый из которых отвечает за свой спектр задач. Это повышает читаемость кода и упрощает его поддержку.

Модуль storage.js отвечает за взаимодействие с браузерным хранилищем LocalStorage. В нем реализованы две основные функции. Функция getData() пытается извлечь данные по ключу memology_data. Если хранилище пустое (пользователь зашел впервые), функция возвращает заранее подготовленный тестовый массив из трех объектов и сразу сохраняет его. Функция saveData(memesArray) принимает массив объектов, преобразует его в текстовый формат JSON посредством метода JSON.stringify() и записывает в память браузера.

Модуль form.js подключается только на странице добавления нового элемента. Скрипт перехватывает событие отправки формы (submit) и блокирует стандартную перезагрузку страницы методом event.preventDefault(). Далее происходит считывание значений из всех полей ввода. Для исключения ввода пустых пробелов применяется метод trim(). Процесс валидации проверяет переменные на пустоту: если хотя бы одно поле не заполнено, функция выводит предупреждение с помощью метода alert() и останавливает выполнение. При успешной проверке создается новый объект, которому присваивается уникальный идентификатор на основе текущего времени (Date.now()). Объект добавляется в общий массив, сохраняется в LocalStorage, после чего пользователь автоматически перенаправляется на главную страницу через window.location.href.

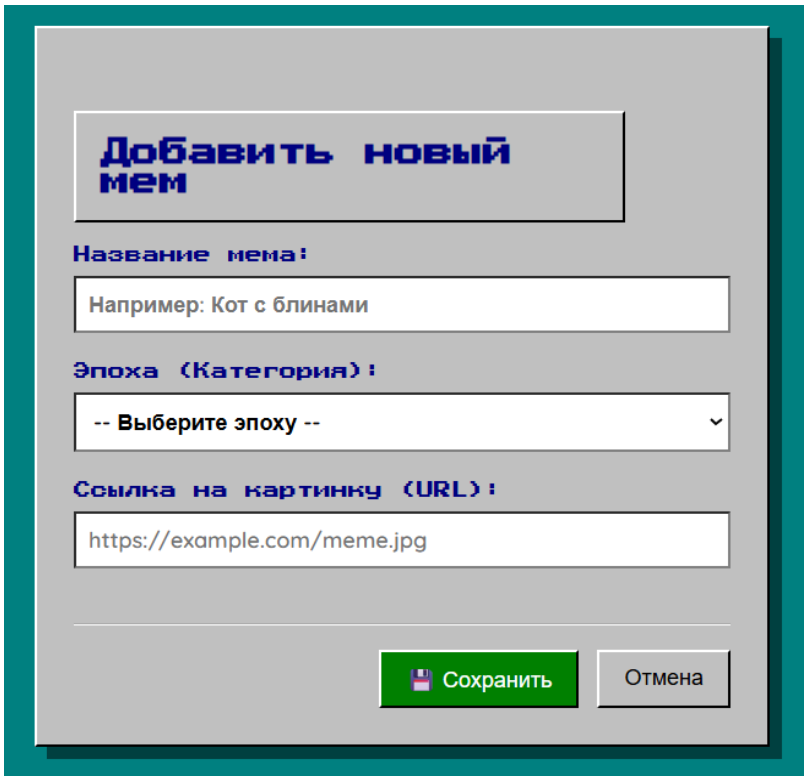

Рисунок 2 - Структура Form

Модуль main.js управляет выводом данных на главной странице. Функция renderItems() очищает контейнер и запускает цикл forEach по массиву сохраненных объектов. Для каждого объекта создается новый HTML-элемент (div), внутрь которого встраивается разметка карточки с заполненными переменными. В этом же модуле реализованы функции взаимодействия с карточками.

Функция toggleStatus(id) находит нужный объект по его идентификатору, меняет логическое значение isLearned на противоположное.

2.4 Тестирование веб-приложения и отладка функционала

Заключительным этапом разработки стало комплексное тестирование готового продукта. Проверка проводилась по нескольким направлениям: функциональное тестирование логики JavaScript, проверка кроссбраузерности и тестирование адаптивной верстки.

Функциональное тестирование показало, что система валидации формы отрабатывает корректно. При попытке отправить пустые поля система выводит alert и не создает пустых объектов в базе данных. Механизм генерации идентификаторов Date.now() успешно справляется с присвоением

уникальных ключей каждому новому элементу. Локальное хранилище данных работает стабильно: при перезагрузке страницы или закрытии браузера все добавленные элементы, а также состояния чекбоксов сохраняются в полном объеме.

Тестирование пользовательского интерфейса (UI) проводилось с использованием инструментов разработчика (DevTools) в браузере Google Chrome. Имитация различных разрешений экрана подтвердила, что CSS-сетка корректно перестраивается. На устройствах с шириной экрана 320px горизонтальная прокрутка отсутствует, элементы не выходят за пределы контейнеров, а размеры кнопок остаются комфортными для нажатия пальцем (touch-friendly дизайн).

В процессе отладки была выявлена проблема с навигационными ссылками на информационных страницах. Из-за дублирования HTML-кода стили кнопок конфликтовали между собой. Проблема была решена путем переработки селекторов в CSS. Кнопкам был оставлен базовый класс retro-button, а их внешний вид начал динамически переопределяться через классы родительских контейнеров (например .header-2000s или .header-modern). Это позволило сохранить единый HTML-каркас, но обеспечить уникальный дизайн кнопок для каждой эпохи. Все выявленные ошибки были успешно устранены, и приложение полностью соответствует заявленным в техническом задании требованиям.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было успешно разработано интерактивное веб-приложение «Мемология», предназначенное для систематизации, коллекционирования и учета цифрового визуального контента. Проект был реализован с использованием базовых веб-технологий (HTML5, CSS3 и нативного JavaScript стандарта ES6+) без привлечения сторонних фреймворков, что позволило создать быстрое и независимое клиентское приложение.

В процессе работы были решены все поставленные на этапе проектирования задачи. Проведенный анализ предметной области позволил выявить ключевые потребности целевой аудитории и сформировать концепцию персонального трекера цифрового контента. На основе этих данных был спроектирован пользовательский интерфейс с уникальным визуальным оформлением, отсылающим к эстетике операционных систем 90-х годов и раннего интернета.

Практическая реализация проекта включила в себя создание семантически корректной разметки страниц и написание масштабируемой таблицы стилей. Применение технологий Flexbox и CSS Grid позволило построить гибкую сетку элементов, а использование медиазапросов обеспечило полную адаптивность веб-приложения для корректного отображения на мобильных устройствах, планшетах и настольных мониторах.

Разработанная на языке JavaScript программная логика обеспечила интерактивность интерфейса. В приложении успешно реализованы функции валидации пользовательского ввода, алгоритмы динамической генерации контента и механизмы управления состоянием элементов (добавление, изменение статуса, удаление). Интеграция с браузерным хранилищем LocalStorage позволила реализовать функционал полноценной локальной базы данных, обеспечивающей надежное сохранение информации между пользовательскими сессиями.

Проведенное на финальном этапе тестирование подтвердило стабильность работы приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- Докинз Р. Эгоистичный ген. Москва: АСТ, 2013. 512 с.
- Флэнаган Д. JavaScript. Подробное руководство. 7-е издание. Санкт-Петербург: Диалектика, 2021. 928 с.
- Хавербеке М. Выразительный JavaScript. Современное веб-программирование. 3-е издание. Санкт-Петербург: Питер, 2019. 480 с.
- Дакетт Дж. HTML и CSS. Разработка и дизайн веб-сайтов. Эксмо, 2021. 512 с.
- Макфарланд Д. JavaScript и jQuery. Исчерпывающее руководство. Эксмо, 2015. 880 с.
- HTML Living Standard. Сообщество WHATWG. URL: https://html.spec.whatwg.org (дата обращения: 10.03.2026).
- CSS Specification. Консорциум W3C. URL: https://www.w3.org/Style/CSS (дата обращения: 10.03.2026).
- MDN Web Docs: JavaScript Reference. Mozilla Foundation. URL: https://developer.mozilla.org/ru/docs/Web/JavaScript (дата обращения: 11.03.2026).
- MDN Web Docs: Window.localStorage. Mozilla Foundation. URL: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage (дата обращения: 12.03.2026).
- MDN Web Docs: CSS Flexible Box Layout. Mozilla Foundation. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout (дата обращения: 12.03.2026).
- MDN Web Docs: CSS Grid Layout. Mozilla Foundation. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout (дата обращения: 12.03.2026).
- MDN Web Docs: CSS Custom Properties. Mozilla Foundation. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties (дата обращения: 13.03.2026).
- Shifman L. Memes in Digital Culture. MIT Press, 2013. 216 p.
- Документация Google Fonts. Typeface Press Start 2P. URL: https://fonts.google.com/specimen/Press+Start+2P (дата обращения: 14.03.2026).
- Документация GitHub Pages. Руководство по развертыванию статических сайтов. URL: https://docs.github.com/en/pages (дата обращения: 15.03.2026).

ПРИЛОЖЕНИЕ А

Листинг программного кода модулей JavaScript

Файл storage.js (Модуль инициализации базы данных и работы с LocalStorage):

```
const defaultMemes = [
    {
        id: 1700000000001,
        title: "Превед, Медвед!",
        category: "2000-е",
        imageUrl: "./images/медвед.gif",
        isLearned: false
    },
    {
        id: 1700000000002,
        title: "Doge (Доге)",
        category: "2010-е",
        imageUrl: "./images/doge-meme.webp",
        isLearned: true
    },
    {
        id: 1700000000003,
        title: "Гарольд, скрывающий боль",
        category: "2010-е",
        imageUrl: "./images/Garold-meme.webp",
        isLearned: false
    }
];

function getData() {
    const data = localStorage.getItem('memology_data');
    if (data) {
        return JSON.parse(data);
    } else {
        saveData(defaultMemes);
```

```
        return defaultMemes;
    }
}

function saveData(memesArray) {
    localStorage.setItem('memology_data', JSON.stringify(memesArray));
}
```

Файл main.js (Модуль управления главной страницей и карточками):

```
const container = document.getElementById('memes-container');
const counter = document.getElementById('total-memes');

function renderItems() {
    const memes = getData();
    container.innerHTML = '';
    counter.textContent = memes.length;

    memes.forEach(meme => {
        const card = document.createElement('div');
        card.className = `meme-card ${meme.isLearned ? 'learned' : ''}`;

        card.innerHTML = `
            <img src="${meme.imageUrl}" alt="${meme.title}">
            <span class="category">${meme.category}</span>
            <h3>${meme.title}</h3>
            <div class="meme-card-actions">
                <label class="checkbox-wrapper">
                    <input type="checkbox" onchange="toggleStatus(${meme.id})"
${meme.isLearned ? 'checked' : ''}>
                    Изучено
                </label>
                <button class="delete-btn"
onclick="deleteMeme(${meme.id})">УДАЛИТЬ</button>
            </div>
        `;
```

```
        container.appendChild(card);
    });
}

function toggleStatus(id) {
    const memes = getData();
    const meme = memes.find(item => item.id === id);
    if (meme) {
        meme.isLearned = !meme.isLearned;
        saveData(memes);
        renderItems();
    }
}

function deleteMeme(id) {
    if (confirm("Вы точно хотите удалить этот мем из базы?")) {
        let memes = getData();
        memes = memes.filter(item => item.id !== id);
        saveData(memes);
        renderItems();
    }
}

document.addEventListener('DOMContentLoaded', renderItems);
```

Файл form.js (Модуль обработки формы добавления данных):

```
const form = document.getElementById('add-meme-form');

form.addEventListener('submit', function(event) {
    event.preventDefault();

    const titleValue = document.getElementById('meme-title').value.trim();
    const categoryValue = document.getElementById('meme-category').value;
    const urlValue = document.getElementById('meme-url').value.trim();
```

```
    if (titleValue === '' || categoryValue === '' || urlValue === '') {
        alert('Ошибка! Пожалуйста, заполните все обязательные поля.');
        return;
    }

    const newMeme = {
        id: Date.now(),
        title: titleValue,
        category: categoryValue,
        imageUrl: urlValue,
        isLearned: false
    };

    const memesArray = getData();
    memesArray.push(newMeme);
    saveData(memesArray);

    window.location.href = 'index.html';
});
```